



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Domain-Specific Language for Polynomial Universal Hash Functions

Master Thesis

Michael Mörschell

June 26, 2026

Advisors: Prof. Dr. Markus Püschel, Tommaso Pegolotti

Department of Computer Science, ETH Zürich

Abstract

Abstract.

Contents

Contents	ii
1 Introduction	1
1.1 Problem Statement	2
1.2 Design Goals	2
1.3 Contributions	3
2 Background	4
2.1 Universal and Delta-Universal Hashing	4
2.2 Polynomial Hash Functions	5
2.2.1 Poly1305	5
2.3 Prime Fields and Crandall Primes	5
2.4 Limb Representations	6
2.5 Prime-Field Arithmetic	7
2.6 Carry Propagation and Limb Realignment	8
2.7 SIMD Model for Polynomial Hashing	9
3 Related Work	10
3.1 Efficient Prime-Field Polynomial Hashing	10
3.2 SPHGen	11
3.3 New Multivariate Polynomial Designs	11
3.4 Prime-Field Hashing Beyond Poly1305	11
4 DSL Design	13
4.1 Design Principles	13
4.2 Types	14
4.3 Core Grammar	14
4.4 Hash Functions and Helper Functions	15
4.5 Expressions and Operations	15
4.6 Reductions	16

4.7	Conditionals	16
4.8	Supported Examples	17
5	Compiler Design	18
5.1	Pipeline Overview	18
5.2	Parsing and AST Construction	18
5.3	Type Checking	19
5.4	Intermediate Representation	20
5.4.1	IR Types	21
5.5	Lowering Ordinary Expressions	21
5.6	Lowering Reductions	22
5.7	Unrolling and Carry Placement	22
5.8	Vectorization	23
5.9	Conditionals	23
5.10	Configuration and Target Parameters	23
6	Code Generation and Runtime Support	25
6.1	Generated C API	25
6.2	Generated Files and Build Integration	26
6.3	Big Integer Representation	26
6.4	Load and Store	27
6.5	Generated Arithmetic	27
6.6	Carry and Full Reduction	28
6.7	Backends	28
6.8	Performance Harness	28
6.9	Optional Formatting	29
7	Correctness and Validation	30
7.1	Reference Semantics	30
7.2	Automatic Boost Tests	31
7.3	Manual Tests	31
7.3.1	Poly1305	31
7.3.2	HKM	31
7.4	Cross-Checks	32
	Bibliography	33
A	Reference Polynomial Definitions	34
B	IR Instruction Reference	36

Chapter 1

Introduction

Universal hash functions are a small but central component in many cryptographic constructions. They are used to turn fixed-key algebraic operations into fast authenticators, most prominently in Carter-Wegman style message authentication codes and in authenticated encryption schemes such as AES-GCM and ChaCha20-Poly1305. The security guarantees of these constructions are information-theoretic at the level of the hash family, while their performance is often close to that of ordinary arithmetic on machine words. This combination makes polynomial universal hash functions an attractive target for both cryptographic design and low-level optimization.

The deployed example that motivates much of the recent literature is Poly1305. It evaluates a polynomial over the prime field defined by $2^{130} - 5$, and it is widely used as part of ChaCha20-Poly1305. Poly1305 also illustrates a broader point: a polynomial hash function is not only a mathematical object. Its performance depends on a dense set of implementation choices, including the prime field, the limb representation of field elements, the multiplication algorithm, the placement of carry propagation, and the vector instruction set of the target machine. Small changes in these choices can change both throughput and the amount of security obtained from a fixed number of input bytes.

Recent work has begun to systematize this design space. Degabriele, Gilcher, Govinden, and Paterson collect implementation techniques for polynomial hashing over prime fields and propose several alternatives to Poly1305 [1]. SPHGen then turns part of this space into a program generator for simple polynomial hash functions over Crandall prime fields [2]. In parallel, new multivariate polynomial designs have been proposed that aim to improve the security-performance trade-off by combining lower-level and higher-level polynomial hash functions [3]. These developments suggest a natural compiler question: can we describe polynomial hash families at a higher level, while still generating efficient low-level code over a range of fields and

instruction sets?

1.1 Problem Statement

The problem addressed in this thesis is the design and implementation of a small domain-specific language (DSL) for polynomial universal hash functions. The language should let a cryptographic designer describe the algebraic structure of a hash function without manually writing the associated field arithmetic, vectorized loops, carry propagation, and test harnesses. The compiler should then lower this description to C code for concrete prime fields and target platforms.

The difficulty is that the useful abstraction boundary is not obvious. A language that exposes only low-level arithmetic parameters is easy to compile, but it does not express the structure of newer polynomial designs. Conversely, a language that hides all implementation choices risks preventing the optimizations that make prime-field hashing fast in practice. The goal of this project is to explore a middle ground: a language expressive enough for the lower-level designs from the multivariate-polynomial literature, but close enough to the implementation model of SPHGen to support vectorization and specialized field arithmetic.

1.2 Design Goals

The compiler developed in this thesis is guided by four design goals.

First, the source language should reflect the algebraic shape of polynomial hash functions. In particular, it should provide direct constructs for finite reductions such as sums, Horner-style polynomial evaluation, and left folds. These constructs are closer to the notation used in cryptographic papers than explicit loops and mutable variables.

Second, the compiler should separate the mathematical description from machine-dependent implementation choices. The same DSL module should be compilable for different Crandall prime fields, multiplication algorithms, and vector targets. The programmer still selects these parameters, but the source program need not be rewritten for every field.

Third, the generated code should expose a stable hash-function API. At the C boundary, hash functions operate on byte buffers and a byte length. Internally, the compiler maps these bytes to field elements according to the selected field size. This is important because changing the field changes the number of message bytes consumed per field element.

Fourth, the implementation should come with practical validation tools. Since this compiler is intended for cryptographic code, a generated implementation

without a convenient correctness story would not be useful. The compiler therefore emits automatic tests based on a high-precision C++ reference implementation, and handwritten tests cover cases where an external reference is available.

1.3 Contributions

This thesis makes the following contributions.

- We design a DSL for describing polynomial universal hash functions over Crandall prime fields. The language supports field elements, indices, buffers, helper functions, hash functions, arithmetic expressions, conditionals, and three reduction constructs.
- We define an intermediate representation (IR) for scalar and vectorized prime-field arithmetic. The IR is structured around explicit instructions, loops, conditionals, loads, stores, carry propagation, and full reductions.
- We implement a C backend for scalar code, ARM NEON, and AVX2. The backend emits field arithmetic for Crandall prime fields, including schoolbook multiplication, Karatsuba multiplication, dedicated squaring, load/store routines, carry propagation, and full reduction.
- We implement compiler lowering for the lower-level polynomial patterns that appear in the new-designs literature, including sums for MMH/SQH/NMH-style constructions, Horner-style evaluation for Poly1305-like functions, and left folds for iterative HKM-style recurrences.
- We generate correctness tests automatically using Boost Multiprecision [4] as a reference, and we add manual validation tests for important special cases such as Poly1305 and HKM.
- We generate a small benchmarking harness for compiled hash functions, allowing the produced implementations to be measured over byte-oriented input sizes.

The current project focuses on the compiler and the generated implementations. The examples from [3] serve to exercise the language and lowering pipeline without turning the compiler into a collection of hand-written implementations.

Chapter 2

Background

This chapter introduces the cryptographic and implementation background needed for the rest of the thesis. We start with universal hashing and polynomial hash functions, then move to prime-field arithmetic and limb representations. Finally, we describe the SIMD layout that is used by the generated vectorized code.

2.1 Universal and Delta-Universal Hashing

Let $\mathcal{R}$ be a key space, $\mathcal{M}$ a message space, and $\mathcal{D}$ an output domain. A keyed hash family is a function

$$H : \mathcal{R} \times \mathcal{M} \rightarrow \mathcal{D}.$$

For a key $r \in \mathcal{R}$, we write $H_r(M)$ for $H(r, M)$. A family is ε -almost universal if, for all distinct messages $M, M' \in \mathcal{M}$,

$$\Pr_{r \leftarrow \mathcal{R}} [H_r(M) = H_r(M')] \leq \varepsilon.$$

In many cryptographic applications one needs the stronger delta-universal variant. If $\mathcal{D}$ is an additive group, H is ε -almost delta-universal if, for all $M \neq M'$ and all $Z \in \mathcal{D}$,

$$\Pr_{r \leftarrow \mathcal{R}} [H_r(M) - H_r(M') = Z] \leq \varepsilon.$$

These definitions are useful because they can be combined with pseudorandom masks or block-cipher calls to build message authentication codes. The resulting construction inherits computational security from the primitive used for masking, while the collision behavior of the hash family is bounded information-theoretically.

2.2 Polynomial Hash Functions

Polynomial hash functions instantiate H by evaluating a polynomial over a finite field. The classical univariate construction interprets message blocks $M_1, \dots, M_n$ as field elements and evaluates

$$H_{\text{poly}}(r, M) = \sum_{i=1}^n M_i r^{n-i} = M_1 r^{n-1} + M_2 r^{n-2} + \dots + M_n.$$

This form is convenient because it can be evaluated online by Horner's rule:

$$T \leftarrow M_1, \quad T \leftarrow rT + M_i \quad \text{for } i = 2, \dots, n.$$

The degree of the polynomial controls the universality bound. The exact bound also depends on the message domain and on how byte strings are encoded as field elements. In practice, the encoding must be injective on the intended message space, or otherwise distinct byte strings could be mapped to the same field-element sequence before the polynomial is even evaluated.

2.2.1 Poly1305

Poly1305 is the best-known prime-field polynomial hash function. It works over the field defined by the prime $2^{130} - 5$. Message bytes are split into blocks and interpreted as field elements; a polynomial is then evaluated at a secret key r . In the complete authenticator, the result is combined with a second secret value s , but the polynomial evaluation itself is the part relevant to this thesis.

Poly1305 is important both as a deployed baseline and as an example of the tension between design and implementation. Its original design choices were influenced by implementation techniques and architectures that are no longer dominant. Later work observes that some of these choices, such as key clamping, no longer provide the same implementation benefit in modern integer and vectorized implementations [1]. This motivates revisiting both the field choice and the implementation strategy.

2.3 Prime Fields and Crandall Primes

This thesis focuses on prime fields of the form

$$\mathbb{F}_p, \quad p = 2^\pi - \theta,$$

where θ is small. Such primes are often called Crandall primes. They are attractive for implementation because powers of 2^π can be reduced cheaply:

$$x \cdot 2^\pi \equiv x \cdot \theta \pmod{2^\pi - \theta}.$$

Thus, when a multiplication produces terms above bit position π , these terms can be folded back into the low limbs by multiplying by θ , usually implemented as a short sequence of shifts and additions when θ has low Hamming weight.

Byte encoding. At the API boundary, messages and keys are byte strings. The compiler maps consecutive chunks of bytes to field elements. For a field parameter π , the natural chunk size used in this project is $\lfloor \pi/8 \rfloor$ bytes. This deliberately ignores any fractional byte of field capacity. For example, the field $2^{116} - 3$ consumes 14 bytes per field element, while $2^{226} - 5$ consumes 28 bytes. The remaining bits below the prime are not filled from a following byte; the next byte belongs to the next field element. This convention keeps the byte-to-field mapping simple and uniform across fields.

2.4 Limb Representations

Machine instructions operate on fixed-width words, while the field elements considered here are typically larger than a single word. We write ω for the machine-word size, for example $\omega = 64$ on the targets used in this thesis. Field elements are represented as vectors of limbs stored in ω -bit words. If the regular limb size is λ , then the base is $B = 2^\lambda$, with $\lambda \leq \omega$. An integer a represented by ℓ limbs has the form

$$a = \sum_{i=0}^{\ell-1} a_i 2^{i\lambda},$$

where each a_i is stored in one machine word. The most significant limb may use fewer bits than the other limbs. We denote its size by

$$\lambda' = \pi - (\ell - 1)\lambda,$$

so that a field element in canonical limb form satisfies $0 \leq a_i < 2^\lambda$ for $i < \ell - 1$ and $0 \leq a_{\ell-1} < 2^{\lambda'}$. The limb count is chosen so that $(\ell - 1)\lambda + \lambda' = \pi$.

Saturation. A representation is saturated if the stored limbs use the full capacity of the machine words, i.e. if $\lambda = \omega$ for the regular limbs. It is unsaturated if $\lambda < \omega$. Unsaturated representations are central to fast polynomial hashing over prime fields: the spare $\omega - \lambda$ high bits can temporarily hold carries, allowing additions and partial products to be accumulated before carry propagation is performed.

Balance. Balanced and unbalanced representations describe a different axis. In a balanced representation, all limbs use the same number of value bits. In an unbalanced representation, the most significant limb uses fewer bits. Since

π is rarely a multiple of the chosen limb size, unbalanced representations arise naturally. They complicate some reductions slightly, but they often avoid increasing the limb count.

Parameter choice. In the compiler, the field parameters π, θ , the target word size ω , and arithmetic bounds determine the selected values of λ, λ' , and ℓ . SPHGen gives bounds for the size of intermediate limbs after operations such as schoolbook multiplication [2]. We use the same kind of criterion when selecting a limb size: λ should be as large as possible, because larger limbs reduce ℓ , but still small enough that the configured arithmetic and carry schedule fit in the available machine words. These values then appear in generated constants such as limb masks, load shifts, and carry-propagation code.

2.5 Prime-Field Arithmetic

Field addition in an unsaturated limb representation is straightforward: corresponding limbs are added independently. The result may not be in canonical form, but as long as the limb bounds are respected it still represents the correct field element. A later carry propagation step restores the limb alignment.

Multiplication. Multiplication is more expensive. Let

$$a = \sum_{i=0}^{\ell-1} a_i 2^{i\lambda}, \quad b = \sum_{j=0}^{\ell-1} b_j 2^{j\lambda}.$$

The ordinary schoolbook product is

$$ab = \sum_{k=0}^{2\ell-2} \left(\sum_{i+j=k} a_i b_j \right) 2^{k\lambda}.$$

A direct product therefore has up to $2\ell - 1$ limbs. For a Crandall prime $p = 2^\pi - \theta$, high terms can be folded back using $2^\pi \equiv \theta \pmod{p}$. When the top limb is unbalanced, powers above π also carry a factor $2^{\lambda-\lambda'}$, which we denote as κ . The generated schoolbook multiplication follows the same principle as the usual folded product:

$$c_k = \sum_{i+j=k} a_i b_j + \theta \cdot 2^{\lambda-\lambda'} \sum_{i+j=k+\ell} a_i b_j, \quad 0 \leq k < \ell,$$

with the exact shifts and masks specialized to λ, λ' , and θ . This formula is best read as the algebraic reason why multiplication can produce an ℓ -limb partially reduced result rather than a $2\ell - 1$ -limb integer.

Squaring. Squaring can be treated as a special case of multiplication. In schoolbook multiplication, the products $a_i a_j$ and $a_j a_i$ are computed separately. For squaring these terms are equal, so the cross terms can be computed once and doubled. This reduces the number of limb multiplications and is useful for constructions such as SQH, which are built around squaring.

Karatsuba. Karatsuba multiplication is an alternative to schoolbook multiplication. The compiler uses one Karatsuba level rather than a recursive implementation. Let

$$m = \lceil \ell/2 \rceil, \quad X = 2^{m\lambda},$$

and split the two operands as

$$a = a_0 + Xa_1, \quad b = b_0 + Xb_1,$$

where a_0, b_0 contain the lower m limbs and a_1, b_1 contain the remaining upper limbs. If ℓ is odd, the upper halves are implicitly zero-extended to the lower-half length when forming sums. The compiler then computes three schoolbook products

$$z_0 = a_0 b_0, \quad z_2 = a_1 b_1, \quad z_1 = (a_0 + a_1)(b_0 + b_1) - z_0 - z_2.$$

The unreduced product is reconstructed as

$$ab = z_0 + Xz_1 + X^2 z_2.$$

After this reconstruction, terms whose limb index is at least ℓ are folded back into the lower ℓ limbs using the same Crandall factor as in the schoolbook case. Equivalently, if d_k denotes the k -th limb coefficient of $z_0 + Xz_1 + X^2 z_2$, the generated partially reduced limbs have the shape

$$c_k = d_k + \theta \cdot 2^{\lambda - \lambda'} d_{k+\ell}, \quad 0 \leq k < \ell,$$

with absent coefficients interpreted as zero. This saves limb multiplications inside the three subproducts, but introduces additional additions, subtractions, temporaries, and usually more register pressure. Whether it is faster therefore depends on the limb count and target architecture.

2.6 Carry Propagation and Limb Realignment

Carry propagation transforms a non-canonical but valid limb representation into one whose limbs again satisfy the chosen bounds. For limbs a_i , a carry round repeatedly extracts

$$t_i = \left\lfloor a_i / 2^\lambda \right\rfloor$$

from regular limbs, adds t_i to the next limb, and masks a_i to λ bits. For the most significant limb the extracted value is

$$t_{\ell-1} = \left\lfloor a_{\ell-1} / 2^{\lambda'} \right\rfloor,$$

and the Crandall identity sends it back to the least significant limb as $t_{\ell-1}\theta$. A full reduction at the end of a hash function additionally handles the case where the represented integer is still at least the field prime $p = 2^\pi - \theta$.

Realignment. The term limb realignment is used for the analogous process after multiplication, where intermediate products occupy wider temporary words. In this thesis, both carry propagation and limb realignment appear in the IR as carry instructions. This is slightly imprecise mathematically, but useful in the compiler: both operations are synchronization points that restore the representation expected by later arithmetic.

Delay strategies. The placement of these operations is an important performance choice. Immediate realignment performs carries as soon as possible. Partially delayed realignment performs carry propagation at selected loop boundaries, for example after a configurable number of unrolled iterations. Fully delayed realignment omits intermediate carries and relies on a final reduction, provided the arithmetic bounds allow it. The latter can be very fast for sums of independent products, but it is not automatically safe for every recurrence.

2.7 SIMD Model for Polynomial Hashing

The vectorized code generated by this project uses SIMD lanes to process several field elements in parallel. The layout is limb-major: a vector register holds the same limb of several independent field elements, one per lane. A complete vectorized field value is therefore represented by several vector registers, one for each limb.

Lane layout. This model has two dimensions. Along the limb dimension, several registers together represent one or more big integers. Along the lane dimension, each register contains several independent field elements. For example, on a two-lane 64-bit NEON target, one vectorized field value contains two field elements processed in parallel. On a four-lane 64-bit AVX2 target, it contains four. This layout is well suited to sums and parallel Horner branches, where different message blocks can be processed independently for part of the computation.

Related Work

This thesis builds on three lines of work: the systematization of polynomial hash implementation techniques, program generation for fast prime-field polynomial hashing, and newer multivariate polynomial hash designs motivated by higher-security applications.

3.1 Efficient Prime-Field Polynomial Hashing

Degabriele, Gilcher, Govinden, and Paterson give a systematic overview of design and implementation choices for polynomial hash functions over prime fields [1]. They separate high-level design choices, such as the polynomial construction and finite field, from implementation choices, such as limb representation, multiplication strategy, and carry propagation. This separation is central to our compiler: the DSL describes the polynomial structure, while compiler settings select the field, target platform, vectorization strategy, multiplication algorithm, and carry schedule.

Their work also emphasizes the importance of delayed carry propagation and delayed limb realignment. Unsaturated limb representations leave enough space in each machine word to accumulate temporary overflow bits. This can reduce the number of sequential carry chains, which are costly because they limit instruction-level parallelism. Our compiler exposes this idea through explicit carry instructions in the IR and through a configurable realignment mode.

The paper also proposes new prime-field designs that revisit Poly1305's design choices on modern hardware. These designs are important context for this thesis because they demonstrate that changing the prime field and the implementation strategy can improve performance, security, or both.

3.2 SPHGen

SPHGen is a program generator for fast polynomial hash functions over Crandall prime fields [2]. It takes field and target parameters as input and emits optimized implementations for vector ISAs. Its generated programs contain specialized load, add, multiply, carry, and main-loop code for the selected field representation and target architecture.

Our implementation uses a similar code-generation setting, but exposes it through a small DSL rather than through fixed polynomial templates. We use Crandall prime fields, unsaturated limb representations, vectorized arithmetic routines, and configurable multiplication algorithms, while letting the source program describe the algebraic structure to be lowered.

The difference is the level of abstraction exposed to the programmer. SPHGen targets a specific class of simple polynomial hash functions. The project in this thesis instead starts from a DSL that can express several patterns.

3.3 New Multivariate Polynomial Designs

The work on new multivariate-polynomial universal hash functions proposes two-level polynomial designs that combine lower-level and higher-level hash functions [3]. At the lower level, the paper studies MMH, SQH, NMH, and HKM constructions. The precise definitions used as reference examples are collected in the appendix. These functions are particularly relevant for this thesis as they serve both as propelling examples and as test subjects for our compiler.

MMH is naturally expressed as a sum of products. SQH replaces products by squarings, incentivizing a dedicated square operation. NMH groups message and key elements pairwise and is also naturally expressed as a sum. HKM is different: it is a sequential recurrence. The definition in the paper appears to contain a typo in the recursive call; throughout this thesis we use the intended recurrence. For generated code, a direct recursive implementation is unattractive, because it introduces function-call overhead and prevents the compiler from controlling carry placement. We therefore express the same recurrence iteratively using a left fold.

3.4 Prime-Field Hashing Beyond Poly1305

The workshop paper on universal hash designs discusses why polynomial hash functions should not be tied to the exact Poly1305 parameter set [5]. Its broader message is that prime-field polynomial hashing remains attractive beyond $2^{130} - 5$, especially when the desired security level, message-length

regime, or target platform differs from the setting for which Poly1305 was originally designed.

Larger fields can give stronger security; smaller or better-aligned fields can improve throughput; and designs with different polynomial structures can trade key size, multiplication count, and parallelism differently. This reinforces the need for tooling that can vary the field and the implementation parameters without rewriting the hash function by hand.

DSL Design

The source language is designed to describe polynomial hash functions close to their mathematical presentation. It is intentionally small. Rather than providing a general-purpose imperative language, it offers expressions, functions, conditionals, buffer accesses, field arithmetic, and a few reduction constructs. The examples in this thesis are motivated by concrete designs, but the language is not hard-wired to them: a programmer can combine the available expression forms to describe other polynomial computations over the selected field.

4.1 Design Principles

The most important design principle is that DSL functions are expression-oriented. A programmer writes the value computed by a function, not an explicit sequence of statements that updates local variables. This mirrors the way polynomial hash functions are usually described in papers: as sums, products, recurrences, and polynomial evaluations over a finite domain.

The language has no user-visible mutable variables. Mutable state appears only after lowering, for example as an accumulator in the IR for a reduction. This restriction simplifies type checking and code generation. It also makes it easier to generate a mathematical reference implementation for testing, since the source program has a direct expression-tree interpretation.

At the same time, the DSL is not purely mathematical notation. It contains buffers and index expressions, because generated hash functions must read bytes from memory. It also distinguishes hash functions from helper functions, because hash functions have a stable C API while helper functions may return field elements or indices internally.

4.2 Types

The language has three user-facing types.

Index. Index values are scalar integer values used for loop bounds, offsets, and length computations.

Field element. Field elements are values in the selected prime field $\mathbb{F}_{2^\pi - \theta}$. The source program does not fix π and θ ; these are compiler parameters.

Buffer. Buffers represent byte arrays passed at the C boundary. Loading from a buffer at field-element index i decodes the corresponding byte chunk into a field element.

The field element type is intentionally abstract in the source language. A DSL program does not mention the number of limbs, the limb width, or the SIMD layout. These are introduced by the compiler after the target field and platform are known.

4.3 Core Grammar

Listing 4.1 gives a compact grammar for the central part of the language. It omits lexical details such as comments and numeric-token syntax, but it captures the structure that matters for the compiler: a module contains functions, every function body is exactly one expression, and reductions are expression forms rather than statements.

Listing 4.1: Core grammar of the DSL.

```

module      ::= function+
function    ::= hashfun | helperfun

hashfun     ::= "hash" ident "(" ident "buffer" ","
                    ident "buffer" ","
                    ident "index" ")" "{" expr "}"

helperfun    ::= "func" ident "(" params? ")" rettype "{" expr "}"
params       ::= paramgroup "(" paramgroup)*
paramgroup   ::= ident "(" ident)* type
rettype      ::= "fieldelement" | "index"
type         ::= "buffer" | "fieldelement" | "index"

expr         ::= comparison
comparison   ::= addsub ("=="|"!="|"<"|"<="|">"|">=") addsub)?
addsub       ::= muldiv ("+"|"−") muldiv)*
muldiv       ::= unary ("*"|"/"|"%" unary)*
unary        ::= "−" unary | power
power        ::= primary ("**" power)?

primary      ::= literal ("as" type)?
                    | ident
                    | ident "[" expr "]"
                    | ident "(" (expr ("," expr)* ")"
                    | "if" expr "{" expr "}" "else" "{" expr "}"
                    | "nctif" expr "{" expr "}" ("else" "{" expr "}")?

```

```

      | reduction
      | "(" expr ")"

reduction ::= sum | horner | foldl
sum       ::= "sum" "[" ident "," expr ":" expr ":" expr "]"
           "{" expr "}"
horner    ::= "horner" "[" ident "," expr ":" expr ":" expr
           "," expr "]" "{" expr "}"
foldl    ::= "foldl" "[" ident "," expr ":" expr ":" expr
           "," ident "=" expr "]" "{" expr "}"

```

4.4 Hash Functions and Helper Functions

The DSL distinguishes between hash functions and helper functions. A hash function is the external entry point. In generated C, every hash function has the same shape:

```

void h(uint8_t *output, uint8_t *key,
      uint8_t *message, size_t len);

```

At this boundary, *len* is a byte length. The compiler converts it to a number of field elements using the field chunk size.

Inside the DSL, the corresponding length parameter denotes a number of field elements. This convention keeps DSL programs independent of the selected field. For example, a sum over $0, \dots, len - 1$ always means a sum over decoded message blocks, even though the number of bytes per block changes with π .

Helper functions are internal functions. They may take DSL values as parameters and return indices or field elements. They are useful for factoring out repeated algebraic structure, but they are not exposed through the stable C hash API.

4.5 Expressions and Operations

Index expressions contain ordinary integer operations such as addition, subtraction, multiplication, division, and remainder. They are used to describe loop bounds and buffer positions.

Buffer loads decode byte chunks into field elements. The load index is zero-based and counts field elements, not bytes. Thus, if the field chunk size is $c = \lfloor \pi/8 \rfloor$, then loading *message*[*i*] reads bytes beginning at offset *ic*. The exact load code is target-dependent, but the source-level meaning is independent of the backend.

The expression language contains unary and binary operators. The unary operator is negation. Binary arithmetic operators are $+$, $-$, $*$, $/$, and $\%$; comparison operators are $=$, $\neq$, $<$, $\leq$, $>$, and $\geq$ in mathematical notation, written

in the concrete syntax as the usual C-like tokens. Arithmetic binary operations require both operands to have the same DSL type. For index operands they denote integer arithmetic. For field-element operands, addition and multiplication denote operations modulo the selected prime. Comparisons return index values, used as conditions.

Exponentiation is intentionally restricted. The compiler recognizes exponent 0, 1, and 2. The case x^2 over field elements lowers to a dedicated square operation.

4.6 Reductions

The DSL contains three reduction forms.

Sums. A sum has an index variable, a range, a step, and a body expression. It denotes the finite sum of the body over the index domain. This construct naturally expresses MMH, SQH, and NMH-style lower-level designs. It also gives the compiler a clear vectorization opportunity: several summands can often be evaluated independently and accumulated in parallel.

Horner reductions. A Horner reduction evaluates a polynomial recurrence. It takes a range, a body expression, and an expression for the multiplier r . The multiplier is evaluated once before the loop and then reused. The recurrence has the form

$$acc \leftarrow r \cdot (acc + e_i),$$

where e_i is the body expression at index i . This construct captures Poly1305-style polynomial evaluation and admits the branch-parallel vectorization strategy described in the literature.

Left folds. A left fold introduces an accumulator name and an initial value. For each index in the range, the body expression computes the next accumulator value from the current index and accumulator. Semantically, it is the usual recurrence

$$acc_0 = init, \quad acc_{j+1} = f(i_j, acc_j).$$

In this compiler, left folds are scalar. They are used for loop-carried designs such as the iterative HKM rewrite, where each output depends on the previous accumulator value.

4.7 Conditionals

The language distinguishes constant-time and non-constant-time conditionals. A regular conditional is intended for cryptographic code paths whose timing

should not depend on secret data. It lowers to code that computes both branches and selects the required result. A non-constant-time conditional, written separately in the DSL, lowers to an ordinary branch and may omit an else branch.

This distinction is important because not every branch in a hash function has the same security meaning. Branching on public information, such as the parity or length of a message, is usually acceptable. Branching on secret data is not. The DSL makes this distinction explicit instead of hiding it in code generation.

4.8 Supported Examples

The implemented language supports the examples used in this thesis: MMH, SQH, NMH, Poly1305-style Horner evaluation, and HKM through an iterative left fold. It also supports helper functions, arithmetic expressions, buffer accesses, and explicit conditionals. These examples exercise the main source constructs, but they are not special cases in the compiler. The language should be read as an expression-oriented core for polynomial hash design, not as a catalogue of four hard-coded hash functions.

Compiler Design

The compiler translates a DSL module into C source files, headers, tests, and optionally a benchmarking harness. Its main stages are parsing, AST construction, type checking, IR generation, and code generation. The central design choice is to make the IR explicit enough to control vectorization and carry placement, while keeping the source language compact.

5.1 Pipeline Overview

The compiler takes three conceptual inputs: a DSL source file, a field description, and a target description. The field description consists of π and θ , defining the prime $2^\pi - \theta$. The target description includes the platform, vectorization mode, lane count, multiplication algorithm, unrolling factor, and delayed-realignment mode.

Generated artifacts. From these inputs the compiler produces generated C sources and headers. If requested, it also emits automatic Boost-based correctness tests and a standalone performance harness. The same DSL source can therefore be compiled repeatedly under different fields and targets. Figure 5.1 summarizes this architecture.

5.2 Parsing and AST Construction

The concrete syntax is described by an ANTLR grammar. ANTLR generates a Python parser, which produces a parse tree for a module. The compiler then visits this parse tree and constructs a smaller abstract syntax tree (AST).

AST shape. The AST mirrors the language rather than the grammar. It contains nodes for modules, functions, expressions, reductions, conditionals, buffer loads, literals, unary operations, and binary operations. This

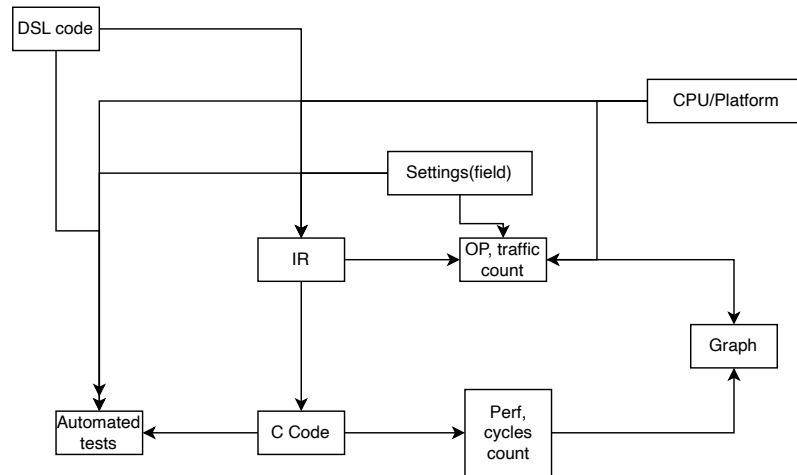


Figure 5.1: Overview of the compiler pipeline from DSL source, field parameters, and target description to generated C code, tests, and performance harnesses.

separation is useful because the grammar contains syntactic details that are irrelevant after parsing, while later compiler phases need a compact representation of the program structure. For example, the grammar has separate precedence levels for addition and multiplication, but the AST only needs a binary-operation node annotated with the operator.

5.3 Type Checking

The type checker assigns DSL types to expressions and verifies that operations are used consistently. Index operations must operate on index values; field operations must operate on field elements; buffer loads produce field elements; and reductions must have bodies of the expected type. Binary arithmetic operators require equal operand types. Comparisons also require equal operand types, but produce an index value so that they can be used as conditions. The exponent operator is checked separately: the exponent must be an index expression, and only the small exponents supported by lowering are accepted.

The checker also distinguishes hash functions from helper functions. Hash functions must have the fixed DSL-level shape corresponding to key buffer, message buffer, and length. Helper functions may be more general, but they cannot use buffers as return values. This keeps the generated C API simple while still allowing source-level factoring.

5.4 Intermediate Representation

The IR is the core internal representation of the compiler. It is not an assembly language, and it is not intended to be lowered directly to machine code. Instead, it is a structured representation of the C code that will be emitted.

Functions and mutation. An IR module contains functions. A function contains statements such as instructions, loops, conditionals, returns, and stores. Instructions produce or update operands. Most temporaries follow an SSA-like discipline, but accumulator updates and in-place carry operations are represented explicitly as updates to an existing destination. This is deliberate: accumulators and carries are precisely the places where the compiler must control mutation and scheduling.

Instruction vocabulary. The instruction set is small. It includes scalar index operations, field operations, memory operations, representation-management operations, and vector helpers. Examples are `add`, `mul`, `square`, `load`, `vload`, `carry`, `vcarry`, `splat`, and `select`. Vectorized instructions are not separate source-language operations. They are introduced by IR generation when the compiler decides to lower an expression over several lanes. A complete instruction summary is given in Chapter B.

Listing 5.1 shows the pretty-printed IR obtained by compiling the NMH module for the field $2^{226} - 5$, using vectorization, schoolbook multiplication, fully delayed limb realignment, and unrolling factor one on the two-lane NEON backend. The example illustrates how the compiler makes the vectorized sum explicit: the main loop uses `vload`, `vadd`, and `vmul` over matrix values, the scalar tail handles the remaining pairwise products, the public odd-length case is represented as an `nctif`, and the only reduction step after the accumulation is the final `full_reduction`.

Listing 5.1: Pretty-printed IR for vectorized NMH in $GF(2^{226} - 5)$ with schoolbook multiplication, fully delayed limb realignment, and unrolling factor one.

```
module nmh

hash nmh(%key: buffer/pod, %message: buffer/pod, %len: index/scalar) -> prime<226, 5>/
vector {
  %0: index/scalar = sub %len, 1
  %1: index/scalar = sub %len, 1
  %2: index/scalar = sub %len, 1
  %3: index/scalar = sub %2, 0
  %4: index/scalar = mod %3, 4
  %5: index/scalar = sub %1, %4
  %6: prime<226, 5>/matrix = const 0
  loop var %i from 0 to %5 step 4 {
    %7: prime<226, 5>/matrix = vload %message, %i, 0, 2
    %8: prime<226, 5>/matrix = vload %key, %i, 0, 2
    %9: prime<226, 5>/matrix = vadd %7, %8
```



```

%10: index/scalar = add %i, 1
%11: prime<226, 5>/matrix = vload %message, %10, 0, 2
%12: index/scalar = add %i, 1
%13: prime<226, 5>/matrix = vload %key, %12, 0, 2
%14: prime<226, 5>/matrix = vadd %11, %13
%15: prime<226, 5>/matrix = vmul %9, %14
%6 = vadd %6, %15
}
%16: prime<226, 5>/vector = horiz_add %6
loop var %i from %5 to %0 step 2 {
  %17: prime<226, 5>/vector = load %message, %i, 0
  %18: prime<226, 5>/vector = load %key, %i, 0
  %19: prime<226, 5>/vector = add %17, %18
  %20: index/scalar = add %i, 1
  %21: prime<226, 5>/vector = load %message, %20, 0
  %22: index/scalar = add %i, 1
  %23: prime<226, 5>/vector = load %key, %22, 0
  %24: prime<226, 5>/vector = add %21, %23
  %25: prime<226, 5>/vector = mul %19, %24
  %16 = add %16, %25
}
%26: prime<226, 5>/vector = const 0
%27: index/scalar = mod %len, 2
nctif %27 {
  %28: index/scalar = sub %len, 1
  %29: prime<226, 5>/vector = load %message, %28, 0
  %26 = copy %29
}
%30: prime<226, 5>/vector = add %16, %26
%30 = full_reduction %30
store %30
}

```

5.4.1 IR Types

The IR uses three representation types: scalar, vector, and matrix. The terminology is implementation-oriented. An index value is scalar. A single field element represented by several limbs is called a vector because it consists of a vector of limb components. Several field elements distributed across SIMD lanes form a matrix, because each limb component is itself a SIMD vector. This naming is internal to the compiler, but it is useful when selecting the generated C representation.

5.5 Lowering Ordinary Expressions

Before reductions are considered, most expression lowering is local. Literals lower to const. Identifiers lower to bound parameters, loop variables, or temporaries from the current compile context. Buffer reads lower to load or vload, depending on the number of lanes requested by the surrounding reduction.

Unary minus is represented as subtraction from zero. Binary operations lower according to their operand type. For index operands, operations such

as `sub`, `div`, `mod`, and comparisons compile to scalar C operators. For field operands, addition and multiplication lower to field arithmetic instructions; when more than one lane is being compiled, the corresponding vectorized instruction is used. Exponentiation is simplified during lowering: x^0 becomes one, x^1 becomes x , and x^2 becomes the dedicated square instruction.

5.6 Lowering Reductions

Reductions are lowered in the IR because they carry most of the optimization structure. They are also the main place where source-level algebraic structure influences the generated loops.

Sums. For sums, the compiler creates an accumulator and emits a loop over the index domain. If vectorization is enabled, the loop step is multiplied by the number of SIMD lanes. Each vectorized iteration evaluates several body instances in parallel and accumulates them in a vectorized field value. A scalar tail handles the remaining elements when the iteration count is not a multiple of the vector width. The body itself may be any expression accepted by the type checker; the compiler does not need to know whether the programmer intended MMH, NMH, or another sum-shaped construction.

Horner reductions. Horner reductions are sequential in their simplest form, but they can be vectorized by evaluating several branches in parallel using precomputed powers of the key. The compiler evaluates the multiplier expression once, constructs the required lane powers, and emits a vectorized main loop plus a scalar tail. This follows the same structural idea as parallel Horner evaluation.

Left folds. Left folds are lowered as scalar loops. The accumulator is explicitly bound in the compile context for the body expression, allowing the body to refer to the previous accumulator value. The compiler supports unrolling for left folds, but it does not vectorize them, because the intended use case has a loop-carried dependency.

5.7 Unrolling and Carry Placement

The unrolling factor has two roles. It reduces loop overhead, and it controls how often carry propagation is inserted in partially delayed mode. In the main unrolled body, several logical reduction steps are emitted before a carry instruction is inserted. The tail loop then handles the remaining iterations and carries at the scalar boundary.

In fully delayed mode, intermediate carry instructions are omitted and the generated function relies on the final full reduction. This can be profitable for reductions whose terms are independent, but it is not a universal transformation. Arithmetic safety depends on the selected field, limb representation, reduction structure, and unrolling factor.

5.8 Vectorization

Vectorization is expressed by compiling selected expressions with more than one lane. The compile context records offsets for loop indices, so that the same source expression can be instantiated for lane 0, lane 1, and so on. For example, a body expression containing *message[i]* becomes several loads from *message[i]*, *message[i + 1]*, etc., packed into a vectorized field value.

The generated code keeps temporaries at full machine width, typically 64 bits per limb. This is conservative, but it keeps addition and multiplication results in a uniform representation. Multiplication code may narrow or mask operands internally where required.

5.9 Conditionals

The IR distinguishes constant-time and non-constant-time conditionals. A constant-time conditional lowers to both branches followed by a select. A non-constant-time conditional lowers to an ordinary branch. The latter is useful for public control flow such as handling even and odd message lengths. The distinction is visible in the IR so that code generation does not have to infer the security intention of a branch from context.

5.10 Configuration and Target Parameters

The compiler configuration fixes the field and machine parameters. Given π , θ , the target platform, and the vectorization mode, the settings code selects a limb arrangement. In particular, it chooses a large limb size that satisfies bounds on intermediate values, following the same motivation as the SPHGen parameter selection: reduce the limb count without overflowing during the configured arithmetic. The configuration also selects schoolbook or Karatsuba multiplication, the unrolling factor, and the delayed-realignment mode.

Supported targets. The current implementation focuses on scalar code, two-lane 64-bit ARM NEON, and four-lane 64-bit AVX2. These targets are sufficient to exercise the main compiler abstractions: scalar arithmetic,

vectorized limb-major layout, platform-specific load/store routines, and different vector widths.

Configuration responsibility. The compiler exposes these choices but does not prove that every combination is safe. Selecting an excessively aggressive unrolling factor or fully delayed carry strategy can exceed the available limb slack. In this sense, arithmetic safety is partly a configuration responsibility. The generated tests are intended to catch such mistakes during development.

Code Generation and Runtime Support

After IR generation, the compiler emits ordinary C code for the selected field and target. This chapter describes the generated API, the byte-to-field mapping, the generated files, and the arithmetic implemented by the backend: data structures for field elements, load/store routines, arithmetic procedures, carry propagation, and loops corresponding to lowered reductions.

6.1 Generated C API

Every DSL hash function is emitted as a C function with the signature

```
void h(uint8_t *output, uint8_t *key,  
      uint8_t *message, size_t len);
```

The parameter *len* is a byte length at the C boundary. The generated function converts it to a number of field elements using the field chunk size $c = \lfloor \pi/8 \rfloor$. The DSL body then sees this derived field-element count.

Buffer convention. The caller is responsible for providing input buffers with enough padding for the generated loads. This convention keeps the inner loops simple: the compiler can compute the number of blocks as a ceiling division and may read slightly past the logical end of the message, as long as the caller has provided safe padding.

Output format. The output buffer is assumed to be large enough to hold the exported field element. The generated store routine writes the final reduced value in little-endian byte order. The number of exported bytes is $\lceil \pi/8 \rceil$, while the number of bytes consumed per input field element is $\lfloor \pi/8 \rfloor$. This asymmetry is intentional: input chunks avoid fractional bytes, while output must be able to represent a field element.

6.2 Generated Files and Build Integration

For a module named m , the compiler emits a header $m.h$, a source file $m.c$, and shared data-structure files for the selected field representation. When automatic tests are enabled, it also emits a C++ test file. When performance generation is enabled, it emits a standalone C benchmarking source for the module.

The surrounding CMake project builds correctness and performance targets from the generated files. This build integration is deliberately thin. The compiler is responsible for producing ordinary C and C++ sources; CMake is responsible for compiling and linking them with handwritten tests and dependencies such as OpenSSL or Boost.

6.3 Big Integer Representation

Generated C code represents a field element as a struct containing one array entry per limb. Scalar field elements use machine-word limbs. Vectorized field elements use SIMD vector types as limb entries. Thus, the same high-level operation can be implemented by iterating over limbs, regardless of whether the limb type is scalar or vectorized.

The generated data structures also define constants derived from the field: the number of limbs, the limb mask, the size of the most significant limb, the input chunk size, and the number of bytes exported by a store. These constants are generated per field, so handwritten C tests and generated code use the same representation.

Listing 6.1 shows a generated data-structure fragment for the field $2^{116} - 3$ on a two-lane NEON target. In this configuration, $\lambda = 18$, $\lambda' = 8$, and $\ell = 7$. The scalar type stores one field element, while the vectorized type stores two field elements in limb-major form.

Listing 6.1: Generated scalar and vectorized field-element data structures.

```
#include <arm_neon.h>

#define FIELD_PI 116ull
#define FIELD_THETA 3ull
#define FIELD_LAMBDA 18ull
#define FIELD_LAMBDA_PRIME 8ull
#define FIELD_LIMBS 7ull
#define FIELD_CHUNK_SIZE 14ull
#define FIELD_EXPORT_BYTES 15ull
#define FIELD_PRIME_HEX "Oxffffffffffffffffffffffffffffd"

typedef struct {
    uint64_t limb0;
    uint64_t limb1;
    uint64_t limb2;
    uint64_t limb3;
    uint64_t limb4;
```

```

    uint64_t limb5;
    uint64_t limb6;
} bigint_t;

typedef struct {
    uint64x2_t limb0;
    uint64x2_t limb1;
    uint64x2_t limb2;
    uint64x2_t limb3;
    uint64x2_t limb4;
    uint64x2_t limb5;
    uint64x2_t limb6;
} vbigint_t;

```

6.4 Load and Store

Loading is field-specific because it maps byte chunks into limbs. For scalar code, the load routine reads consecutive bytes, assembles them into an integer, and splits the result into limbs. For vectorized code, the generated load routine assembles several field elements at once, one per SIMD lane.

The important subtlety is that a loaded input field element contains only $\lfloor \pi/8 \rfloor$ bytes of data. If π is not divisible by eight, the remaining high bits of the field are not filled by consuming another byte. This convention matches the source-language view that buffer indices count field elements of a fixed byte chunk size.

Stores perform the reverse operation after a full reduction. The final field element is exported to bytes. The store operation is part of the IR for hash functions, so returning a field element from a hash does not require an extra wrapper call around generated code.

6.5 Generated Arithmetic

Addition is emitted as component-wise limb addition. In unsaturated representations, this is safe as long as the subsequent carry schedule keeps limb values within bounds.

Multiplication and squaring. Multiplication is emitted either as schoolbook multiplication or as Karatsuba multiplication. The schoolbook variant computes limb products and folds high terms back using the Crandall reduction identity. The Karatsuba variant reduces the number of limb multiplications at the cost of additional additions and higher register pressure. Which variant is faster depends on the field size and target.

Squaring has a dedicated implementation. It exploits symmetry of the product to avoid computing both $a_i a_j$ and $a_j a_i$ separately. The vectorized backends contain corresponding squaring code for NEON and AVX2.

For multiplication, the generated code follows the folded schoolbook formula from Section 2.5. Each output limb accumulates ordinary products whose indices sum to that limb, plus wrapped products whose indices exceed the field size and are multiplied by the Crandall factor. The generated code is specialized enough that the loops are usually unrolled in the emitted C.

6.6 Carry and Full Reduction

Generated carry code propagates overflow bits from one limb to the next. The carry out of the most significant limb is multiplied by θ and folded into the least significant limb. For vectorized code, the same operation is performed lane-wise using SIMD shifts, masks, and additions.

A full reduction is emitted at the end of hash functions. It handles edge cases where carry propagation alone leaves a value in the range above the prime. This final step is more expensive than an ordinary carry, but it is outside the main loop and therefore has less impact on throughput for long messages.

6.7 Backends

The scalar backend is the simplest backend and serves two roles. It is useful in its own right, and it provides a baseline for measuring vectorization speedups.

Vector backends. The NEON backend targets two 64-bit lanes. It is useful for ARM systems where two field elements can be processed in parallel. The AVX2 backend targets four 64-bit lanes. It follows the same limb-major layout, but the wider lane count changes the generated loads, lane-power generation, and vectorized loop structure.

6.8 Performance Harness

The compiler can emit a small benchmark program for a generated hash function. The harness allocates key, message, and output buffers, calls the generated hash function for a range of byte lengths, and reports elapsed cycles per call. It treats the generated hash function as a black box: it depends only on the C API, not on the internal IR or on any static analysis.

The purpose of the harness is practical. It lets the same DSL module be measured under different fields, targets, multiplication algorithms, and carry strategies. The evaluation chapter will use this machinery to compare the generated implementations.

6.9 Optional Formatting

The compiler includes an optional formatting pass for generated C and C++ sources. Formatting is not part of the semantic pipeline, but it makes generated files easier to inspect, debug, and compare against handwritten tests. Since the generated code is intended to be fast but not necessarily readable, this convenience is useful despite being orthogonal to performance.

Correctness and Validation

Cryptographic code generation needs a clear correctness target. In this project, correctness means that the generated C implementation agrees with the mathematical meaning of the DSL program for the selected field and configuration. The compiler does not currently provide a formal proof for every generated program. Instead, it combines a reference semantics, automatic randomized tests, and handwritten cross-checks for important examples.

7.1 Reference Semantics

The reference semantics interprets a DSL program after API-level decoding has turned byte buffers into sequences of field elements. Index expressions denote integer values. Buffer expressions such as *message[i]* denote the *i*-th decoded field element from the corresponding input buffer. Field operations denote addition, multiplication, and squaring in $\mathbb{F}_{2^n-\theta}$. Unary and binary expression nodes are interpreted compositionally, so the meaning of a complex expression is determined by the meanings of its subexpressions and operators.

Reductions have their usual mathematical meanings. A sum denotes a finite summation over its index domain. A Horner reduction denotes repeated application of its polynomial recurrence. A left fold denotes a recurrence over an explicitly named accumulator. A hash function denotes the final field element exported to the output buffer.

This semantics is intentionally independent of limb representations, carry schedules, and vectorization. Those are implementation choices. The generated code is correct if, after decoding inputs according to the generated C API, it produces the same field element as this mathematical interpretation.

7.2 Automatic Boost Tests

For automatic validation, the compiler emits a C++ test file using Boost Multiprecision [4]. The generated reference code is scalar and intentionally not optimized. Its purpose is not speed, but clarity: it evaluates the same DSL expressions using arbitrary-precision integers and reduces modulo the selected prime.

Each automatic test generates randomized key and message data, calls the generated C implementation, calls the Boost reference implementation, and compares the exported outputs. Since the tests are generated from the AST, they can be produced for arbitrary DSL modules that fit the supported language fragment.

This approach is not a proof, but it is effective for development. It catches errors in lowering, load/store code, carry placement, multiplication routines, full reduction, and API-level byte handling. It is also useful when experimenting with aggressive unrolling or delayed carry settings, because invalid configurations tend to fail quickly on randomized inputs.

7.3 Manual Tests

Some examples deserve additional tests beyond the generic Boost oracle.

7.3.1 Poly1305

The Poly1305 DSL module implements the polynomial part of Poly1305. The complete Poly1305 authenticator adds a second key component s after polynomial evaluation. In this project, the DSL test fixes or omits this final addition and compares the generated polynomial output against OpenSSL in the corresponding setting.

This test is useful because OpenSSL is a well-regarded independent implementation. It validates not only field arithmetic, but also the byte-level interpretation of Poly1305 blocks. The omission of the final s addition is deliberate: that addition is outside the prime-field computation modeled by the DSL.

7.3.2 HKM

HKM is tested in two forms. One module follows the corrected recursive recurrence from the new-designs paper. A second module implements the same recurrence iteratively using a left fold. The manual test compares both generated implementations on randomized inputs.

This test serves two purposes. It records the correction of the apparent typo in the paper's recursive definition, and it validates the iterative rewrite used

for performance. The iterative version is the one intended for benchmarking, since it avoids recursive generated C.

7.4 Cross-Checks

We validate generated code in three ways: generated Boost references, manual OpenSSL checks, and targeted tests for load/store and arithmetic routines. Together, these tests provide confidence that the compiler implements the intended semantics for the examples considered in this thesis.

The generated tests are also part of the configuration workflow. They are run for the selected field, representation, unrolling factor, and delayed-realignment mode.

Bibliography

- [1] J. P. Degabriele, J. Gilcher, J. Govinden, and K. G. Paterson, “SoK: Efficient design and implementation of polynomial hash functions over prime fields,” in *2024 IEEE Symposium on Security and Privacy*, 2024, pp. 3128–3146.
- [2] T. Pegolotti, J. Gilcher, K. G. Paterson, and M. Püschel, “SPHGen: A program generator for fast polynomial hash functions,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026, To appear.
- [3] J. Gilcher et al., *New designs of multivariate-polynomial universal hash functions*, Manuscript, 2026.
- [4] Boost. “Boost C++ Libraries.” Version 1.90.0, Accessed: Jun. 3, 2026. [Online]. Available: <https://www.boost.org/>
- [5] J. P. Degabriele, J. Gilcher, J. Govinden, and K. G. Paterson, *Universal hash designs for an accordion mode*, NIST Workshop on the Requirements for an Accordion Mode Cipher, 2024.

Appendix A

Reference Polynomial Definitions

This appendix collects the lower-level polynomial definitions used as reference examples throughout the thesis. They are not built into the compiler. Rather, they are small benchmarks for the expressiveness of the DSL: the same language constructs can be combined to describe other polynomial expressions.

Let $M = (M_1, \dots, M_B)$ be a sequence of field elements in $\mathbb{F}_p$, where $p = 2^\pi - \theta$.

MMH. For keys $r_1, \dots, r_{B-1}$, the MMH-style lower-level hash is

$$\text{MMH}(r, M) = \sum_{i=1}^{B-1} M_i r_i + M_B.$$

It is represented in the DSL by a sum of products plus the final message element.

SQH. For keys $r_1, \dots, r_{B-1}$, the SQH-style hash replaces multiplication by squaring:

$$\text{SQH}(r, M) = \sum_{i=1}^{B-1} (M_i + r_i)^2 + M_B.$$

This is the motivating example for a dedicated square operation in the IR and backend.

NMH. For keys $r_1, \dots, r_B$, the NMH-style hash groups elements pairwise:

$$\text{NMH}(r, M) = \sum_{i=1}^{\lfloor B/2 \rfloor} (M_{2i-1} + r_{2i-1})(M_{2i} + r_{2i}) + (B \bmod 2)M_B.$$

The final term is present only when B is odd.

HKM. The HKM definition is recursive. The paper version appears to contain a typo in the recursive call; the intended recurrence is the one shown here. For $B = 1$,

$$\text{HKM}((r_1), (M_1)) = r_1 + M_1.$$

For larger B , let $h = \lfloor B/2 \rfloor$. Then

$$\begin{aligned} \text{HKM}((r_i)_{i=1}^{h+1}, (M_i)_{i=1}^B) &= \text{HKM}((r_i)_{i=1}^h, (M_i)_{i=1}^{2h-1}) \\ &\quad \cdot (r_{h+1} + M_{2h}) + (B \bmod 2)M_B. \end{aligned}$$

For generated code we use an equivalent iterative formulation expressed as a left fold, because this avoids recursive generated C and exposes carry placement to the compiler. Using zero-based indices, the iterative version used by the DSL module is:

$$\begin{aligned} A_0 &= r_0 + M_0, \\ A_{j+1} &= A_j \cdot (r_{j+1} + M_{2j+1}) + M_{2j+2}, \quad 0 \leq j < \left\lfloor \frac{B-1}{2} \right\rfloor. \end{aligned}$$

For odd B , the final value is $A_{\lfloor (B-1)/2 \rfloor}$. For even B , one additional multiplication is required:

$$A_{\lfloor (B-1)/2 \rfloor} \cdot (r_{B/2} + M_{B-1}).$$

This is exactly the shape expressed by the `foldl` implementation: each loop iteration consumes two message elements and one new key element, and the even-length case is handled after the fold.

Appendix B

IR Instruction Reference

This appendix summarizes the IR instructions used by the compiler. Instructions operate on IR operands and produce or update an IR temporary. The IR type of the result determines whether an instruction is scalar, a single field element, or a vectorized collection of field elements.

const Creates a constant operand. Used for integer constants and zero-initialized field elements.

copy Copies one operand into an existing temporary. Used primarily for accumulator updates and branch results.

call Calls a helper function and stores its return value.

Index arithmetic `add`, `sub`, `mul`, `div`, and `mod` compute scalar arithmetic on index operands. For field operands, `add` and `mul` denote field addition and field multiplication.

Comparisons `eq`, `neq`, `lt`, `le`, `gt`, and `ge` compare scalar values. They return index values used as conditions.

select Constant-time selection between two already computed operands, controlled by a scalar condition.

load Loads one field element from a byte buffer and decodes it into scalar limb representation.

vload Loads several consecutive field elements from a byte buffer and decodes them into vectorized limb representation.

square Computes a scalar field square using the dedicated squaring algorithm.

carry Performs scalar carry propagation and partial Crandall reduction on one field element.

`full_reduction` Performs the final scalar reduction that ensures the stored field element is below the field prime.

`vadd` Adds vectorized field elements lane-wise.

`vmul` Multiplies vectorized field elements lane-wise.

`vsquare` Squares vectorized field elements lane-wise.

`vcarry` Performs carry propagation lane-wise on vectorized field elements.

`splat` Replicates a scalar field element into every SIMD lane.

`lane_powers` Builds a vectorized field element whose lanes contain specified powers or multiples of a scalar field element. This is used by vectorized Horner lowering.

`horiz.add` Horizontally adds the lanes of a vectorized field element, producing one scalar field element.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):

First name(s):

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date

Signature(s)

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.